



Week 4: Forward and Inverse Kinematics

TURTLE Incubator - SO-ARM Software Stack

Introduction to Kinematics

- Position vs motion, kinematics vs dynamics

Forward Kinematics

- Joint space, transformation chains, DH parameters

Inverse Kinematics

- Problem definition, multiple solutions, workspace limits

Singularities

- What they are, why they damage hardware, how to avoid them

SO-ARM Application

- Live FK, IK solver, limit validation, error measurement

Demonstration + Hands-On + Deliverable

- Build and run the complete FK/IK pipeline on real hardware

By end of session your script will:

- 1 Read live joint angles from the SO-ARM
- 2 Run FK to compute where the end-effector is in 3D space (x, y, z)
- 3 Accept a target coordinate from the user
- 4 Run IK to find joint angles that reach that target
- 5 Validate the solution against joint limits
- 6 Send the command to the arm and watch it move
- 7 Run FK again on the actual final position and report the error in mm

This is the exact loop inside every real robotic arm controller.

Industrial arms run this at 500-1000 Hz. You will run it once and understand every line.

Introduction to Kinematics

You already know how to describe where things are (Week 3 - frames, transforms, URDF).
This week you learn how to get the arm to a specific place.

Position

- A static snapshot - where is the end-effector given these joint angles?
- Defined by coordinates (x, y, z) and orientation in space

Motion

- How joint angles change over time to move from A to B
- Motion planning (Week 8) handles this - kinematics is the math behind each snapshot

Kinematics answers two questions:

- Given angles, where is the tip? (FK) | Given a target, what angles do I need? (IK)

These two words get confused constantly. They are not the same thing.

Kinematics

- Studies position, velocity, and acceleration - ignores forces and masses
- "Where will the arm tip be if I rotate joint 3 by 45 degrees?"
- Pure geometry and linear algebra - this is what we cover today

Dynamics

- Studies forces, torques, inertia, and gravity - much more complex
- "How much torque does motor 2 need to hold that position against gravity?"
- Required for advanced control and feedforward compensation

For now: kinematics only. Dynamics comes later when we build controllers.

Forward Kinematics

Joint Space vs. Cartesian Space

Two ways to describe your arm:

Joint Space

- A list of angles - one per motor
- What motors actually control
- Hard to visualize in 3D

Cartesian Space

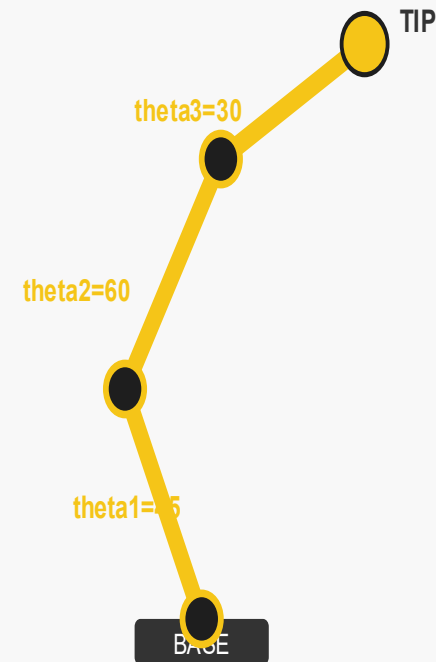
- A position (x, y, z) in meters from the base
- How humans naturally think about targets
- Requires FK/IK to command

FK converts joint space
to Cartesian space.

IK converts Cartesian space
back to joint space.

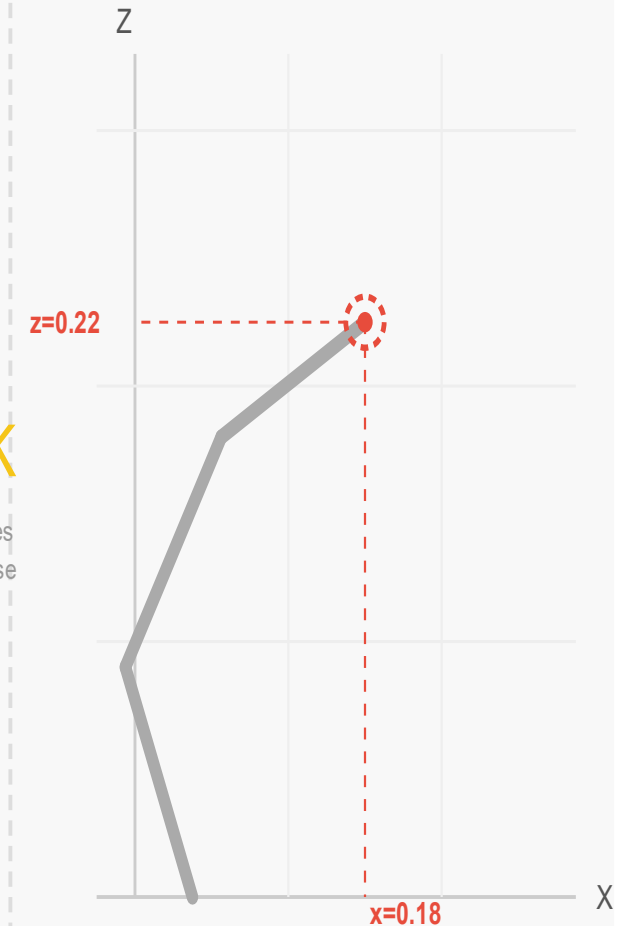
Joint Space

$\theta = [\theta_1, \theta_2, \theta_3]$



Cartesian Space

position = (x, y, z)



FK
angles
to pose

The two directions:

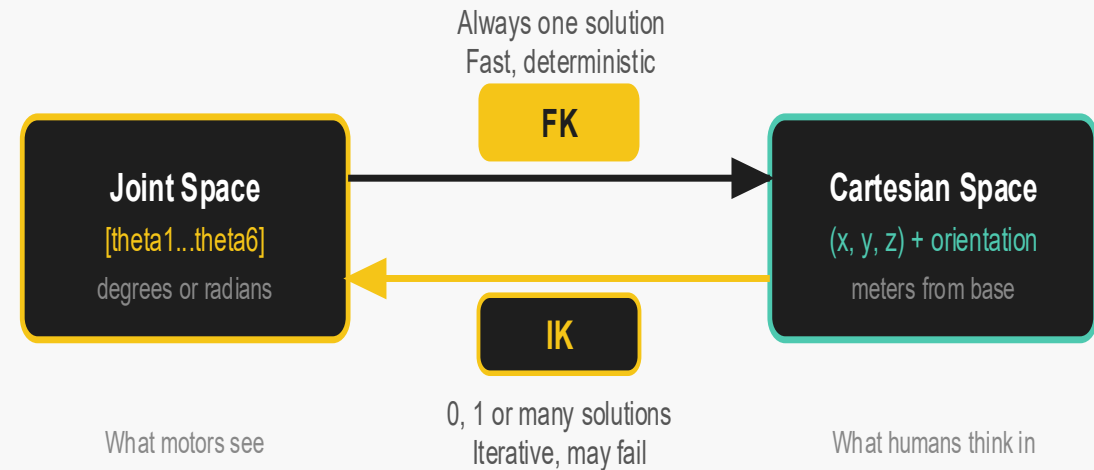
FK (easy)

- Angles in, position out
- Always one unique answer
- Fast and deterministic

IK (hard)

- Target position in, angles out
- May have 0, 1, or many solutions
- Iterative - may fail at workspace boundary

You need both in every real controller.



FK converts joint angles to a position.
IK converts a target position to joint angles.

You need both in every real robot controller.

FK is chaining the transforms you learned in Week 3:

Each joint adds one transform:

- T_i = Rotation about joint axis x Translation along link

Chain them all together:

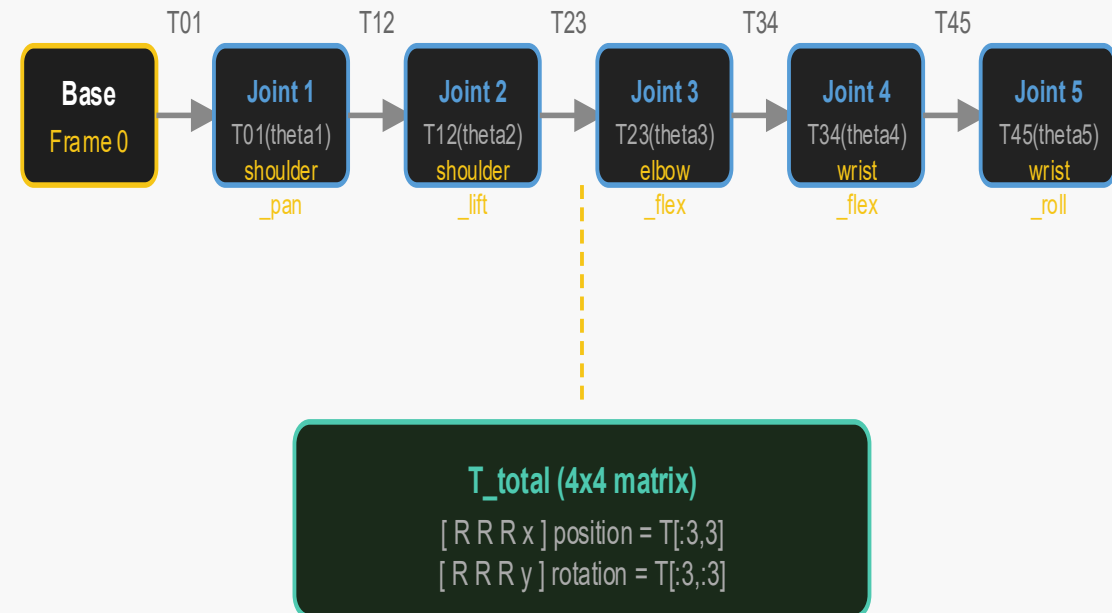
- $T_{total} = T_{01} \times T_{12} \times T_{23} \times T_{34} \times T_{45} \times T_{56}$

The result T_{total} is a 4x4 matrix:

- Top-left 3x3 = end-effector rotation
- Right column = (x, y, z) position

ikpy builds this chain from your URDF automatically.

$$T_{total} = T_{01} \times T_{12} \times T_{23} \times T_{34} \times T_{45} \times T_{56}$$



DH is the standard convention to define each joint transform with just 4 numbers:

- a (link length) - distance along x-axis from one joint to the next
- d (link offset) - distance along z-axis between consecutive frames
- α (link twist) - angle between z-axes of consecutive frames (fixed by geometry)
- θ (joint angle) - THE VARIABLE - this is what the motor controls each cycle

For the SO-ARM100, the DH table has 6 rows - one per joint.

- a and α come from the URDF you read in Week 3. θ changes every control cycle.

$$T_i(\theta_i) = \text{Rot}_z(\theta_i) \times \text{Trans}_z(d_i) \times \text{Trans}_x(a_i) \times \text{Rot}_x(\alpha_i)$$

Inverse Kinematics

IK answers the opposite question from FK:

- Given a target position (x, y, z) for the end-effector, what joint angles get me there?

Input:

- A desired end-effector pose - position and orientation in Cartesian space

Output:

- A set of joint angles $[\theta_1 \dots \theta_6]$ that achieve that pose

IK is not always solvable, and when it is, there may be multiple solutions.

We use numerical solvers (not closed-form algebra) for 6-DoF arms like the SO-ARM.

FK is a function: angles to pose. IK is inverting that function. That inversion is non-trivial.

1 Non-linearity

- FK equations involve $\sin()$ and $\cos()$ - you cannot solve a simple linear system

2 Multiple solutions

- There can be 0, 1, 2, or infinitely many joint configurations that reach the same target

3 Redundancy and limits

- A 6-DoF arm has 6 unknowns and 6 equations - just enough to specify full pose
- But joint limits mean many mathematical solutions are physically invalid

Solution: numerical IK. Start from current joint angles and iteratively converge on a solution.

Multiple solutions - for most targets a 6-DoF arm has several valid configs:

- Elbow up vs. elbow down - two ways to reach the same point
- Wrist flip - same orientation rotated 180 degrees
- ikpy picks the solution closest to the initial_position you provide (current joint angles)

Workspace limits - not every point is reachable:

- Max reach - arm fully extended in one direction
- Min reach - arm folded on itself, inner zone is unreachable
- Dexterous workspace - subset of reachable space where ANY orientation is possible

If IK returns NaN or garbage: the target is outside the workspace or joint limits.

Singularities

What Are Singularities?

A singularity is a configuration where the arm loses one or more degrees of freedom.

Mathematically:

- The Jacobian J (maps joint velocities to Cartesian velocities) becomes rank-deficient
- $\det(J) = 0$ - the matrix is not invertible

Physically:

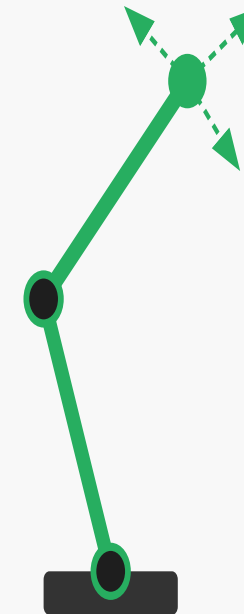
- Two or more joint axes align - multiple joints produce the exact same motion
- Arm cannot move in certain Cartesian directions no matter what joints do

Common types:

- Elbow singularity - arm fully extended
- Wrist singularity - joints 4 and 6 axes align
- Shoulder singularity - wrist center on shoulder axis

Normal Config

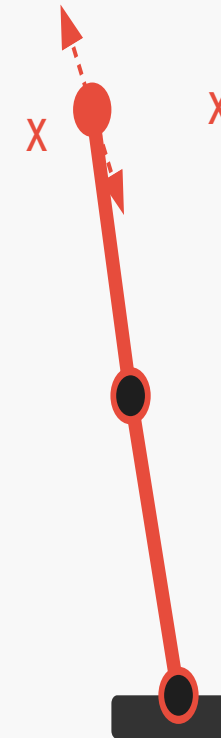
Elbow bent - full control



Can move in all directions

Singular Config

Elbow straight - DoF lost



Stuck - cannot move sideways

Joint velocity spikes near singularities - motors can stall or strip gears

Singularities cause real hardware damage - not just a math problem:

Joint velocity spikes

- Near a singularity, tiny Cartesian movements require massive joint angle changes
- Motors commanded to spin thousands of degrees/sec - stall, overheat, strip gears

Loss of path accuracy

- Arm cannot maintain a straight Cartesian path through a singularity
- Trajectory tracking breaks - arm jerks or freezes

How to handle them:

- Plan paths that avoid singular configurations entirely
- Damped Least Squares (DLS) - adds regularization to prevent velocity spikes
- Monitor $\det(J)$ at runtime - slow down or stop if approaching zero

SO-ARM Application

The SO-ARM100 is a 6-DoF serial chain arm. All joints are revolute (rotation only).

Joint layout - base to tip:

- 1 shoulder_pan - rotates the whole arm left/right about a vertical axis
- 2 shoulder_lift - pitches the upper arm up/down
- 3 elbow_flex - bends the forearm up/down
- 4 wrist_flex - pitches the wrist up/down
- 5 wrist_roll - rolls the wrist about the forearm axis
- 6 gripper - opens/closes (does not affect end-effector pose)

Joints 1-5 determine end-effector pose. ikpy ignores joint 6 in kinematics.

This is the full loop you are building:

Step 1 Get target from user

- `input()` call for (x, y, z)

Step 2 Solve IK

- `chain.inverse_kinematics()`

Step 3 Validate limits

- Reject if any angle out of range

Step 4 Send to arm

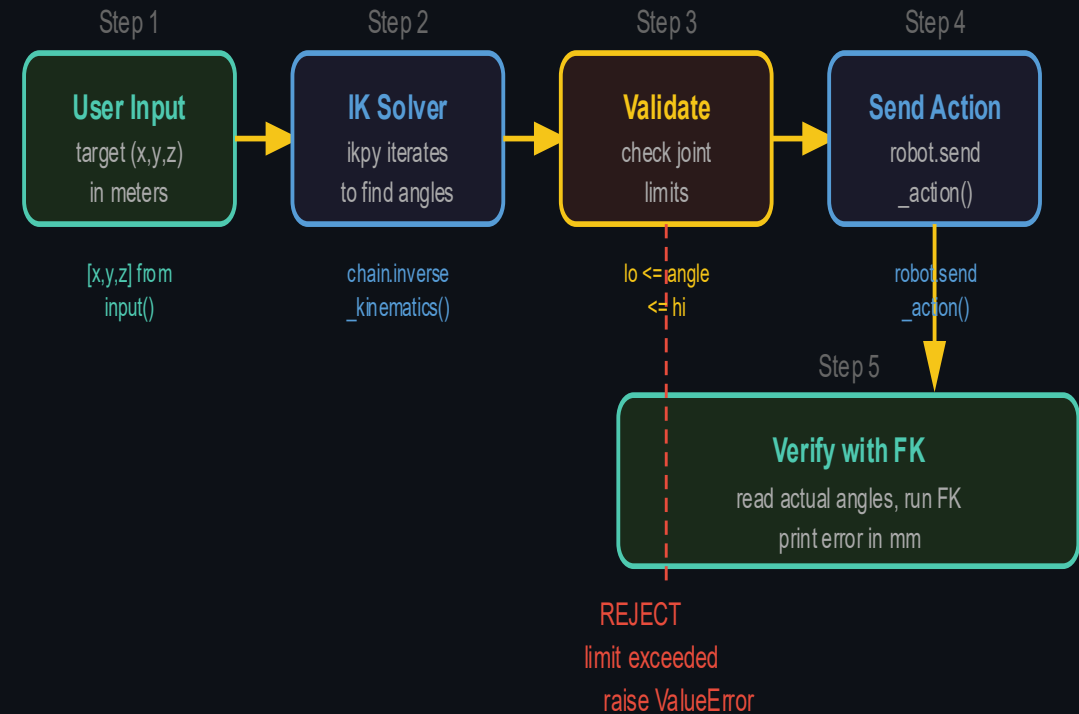
- `robot.send_action()`

Step 5 Verify with FK

- Read actual angles, compute error in mm

The Complete FK / IK Pipeline

This is what your hands-on script must implement end to end



```
from ikpy.chain import Chain
import numpy as np

# Build chain from URDF (reads link lengths and joint axes automatically)
chain = Chain.from_urdf_file('urdf/so100.urdf',
    active_links_mask=[False, True, True, True, True, True, False])

# Read current joint angles from robot (returns degrees)
angles_deg = robot.read('Present_Position', robot.motor_names[:5])
angles_rad = np.deg2rad(angles_deg)

# Run FK - prepend/append 0 for base and gripper fixed links
T = chain.forward_kinematics([0] + list(angles_rad) + [0])

# Extract position from the 4x4 result matrix
position = T[:3, 3]          # [x, y, z] in meters from base
rotation = T[:3, :3]         # 3x3 rotation matrix of end-effector
print(f'End-effector position: {position}')
```

IK Solver - Find Joint Angles for a Target

```
# Define target position in meters from the base origin
target_position = [0.15, 0.0, 0.20]    # 15cm forward, centered, 20cm up

# Read current position to use as starting guess for IK
current_rad = np.deg2rad(robot.read('Present_Position', robot.motor_names[:5]))

# Solve IK - starts from current angles, finds closest valid solution
solution = chain.inverse_kinematics(
    target_position,
    initial_position=[0] + list(current_rad) + [0]
)

# Extract joint angles (skip first and last - fixed links)
joint_angles_rad = solution[1:-1]
joint_angles_deg = np.rad2deg(joint_angles_rad)
print(f'IK solution (deg): {joint_angles_deg}')
```

```
# Joint limits from sol100.yaml
LIMITS = {'shoulder_pan': (-150,150), 'shoulder_lift': (-90,90),
          'elbow_flex': (-150,150), 'wrist_flex': (-90,90), 'wrist_roll': (-180,180)}

# Validate solution against limits
action = {}
for name, angle, (lo, hi) in zip(robot.motor_names[:5], joint_angles_deg, LIMITS.values()):
    if not (lo <= angle <= hi):
        raise ValueError(f'{name}: {angle:.1f}deg out of range [{lo}, {hi}]')
    action[name] = angle

robot.send_action(action)  # arm moves to IK solution

# Verify with FK - compute positional error
actual_rad = np.deg2rad(robot.read('Present_Position', robot.motor_names[:5]))
T_actual = chain.forward_kinematics([0] + list(actual_rad) + [0])
error_mm = np.linalg.norm(T_actual[:3,3] - target_position) * 1000
print(f'Position error: {error_mm:.1f} mm')
```

Common Mistakes

3 Errors You Will Definitely Hit

Every student hits these. Read this before starting the hands-on.

Error 1 - Radians vs. Degrees mismatch

- The robot reports angles in DEGREES. ikpy expects RADIANS.
- Always convert: `angles_rad = np.deg2rad(angles_deg)` before passing to ikpy
- And convert back: `joint_angles_deg = np.rad2deg(solution[1:-1])` before `send_action()`

Error 2 - IK returns NaN or very large numbers

- Target is outside the workspace - the arm physically cannot reach it
- Try targets close to the arm: x in [0.05, 0.25], y in [-0.15, 0.15], z in [0.05, 0.30]

Error 3 - Wrong active_links_mask

- The SO-ARM URDF has 7 links but only 5 are active joints for IK
- Always use: `[False, True, True, True, True, True, False]`

Demonstration

Watch every step - you will repeat this in the hands-on.

What we will show:

- 1. Read current joint angles and run FK - print live (x, y, z)
- 2. Rotate joint 1 manually, re-run FK, observe x/y change but z stays similar
- 3. Define target [0.15, 0.0, 0.20] and solve IK - print solution angles
- 4. Validate against limits - show what happens when a limit is violated
- 5. Send to arm - watch it move to target position
- 6. Run FK on actual angles - print error in mm and explain why it is non-zero
- 7. Request an out-of-workspace target - show IK failure gracefully

Hands-On

Goal: script reads joint angles and prints live end-effector (x, y, z) every 0.5 seconds.

1. Set up the chain

- `chain = Chain.from_urdf_file('urdf/so100.urdf',
active_links_mask=[False,True,True,True,True,True,False])`

2. Write the loop

- `import time`
- `while True:`
- `a = np.deg2rad(robot.read('Present_Position', robot.motor_names[:5]))`
- `T = chain.forward_kinematics([0]+list(a)+[0])`
- `print(f'EE: {T[:3,3]}') ; time.sleep(0.5)`

3. Move joints manually (power off first) and verify coordinates update correctly

Goal: accept a user target, solve IK, move the arm, report the error.

1. Prompt for target

- `x = float(input('x(m): ')) ; y = float(input('y(m): ')) ; z = float(input('z(m): '))`

2. Solve IK from current position

- `current = np.deg2rad(robot.read('Present_Position', robot.motor_names[:5]))`
- `sol = chain.inverse_kinematics([x,y,z], initial_position=[0]+list(current)+[0])`

3. Validate against limits, send, then verify

- Use the LIMITS dict, call `robot.send_action()`, then run FK on actual angles
- Print: Target vs Actual vs Error in mm

Test with at least 3 different targets. Try one outside the workspace intentionally.

Deliverable

Demonstrate all four checkpoints to a lab lead before leaving:

1 Live FK running

- Script reads joint angles and prints (x, y, z) continuously. Move a joint - coordinates update.

2 IK solver working

- Enter a target coordinate. Script prints valid joint angles. Explain which solution was chosen.

3 Arm physically reaches the target

- Arm moves to the requested (x, y, z). Limit validation runs without errors.

4 Error measured and explained

- FK on actual positions reports error in mm. You explain one reason the error is non-zero.